

✓ 1. Install Dependencies

First, we'll install all the required libraries. This cell will install them in your Colab environment. When you set up your local environment, you'll install them using `pip install -r requirements.txt` after creating the file with the content provided in the next step.

```
%%capture
pip install fastapi uvicorn streamlit diffusers torch Pillow requests
```

✓ 2. requirements.txt

Create a file named `requirements.txt` in your project's root directory and paste the following content into it. This file lists all the dependencies for your project.

```
print("""fastapi
uvicorn
streamlit
diffusers
torch
Pillow
requests""")
```

```
fastapi
uvicorn
streamlit
diffusers
torch
Pillow
requests
```

```
%%writefile requirements.txt
fastapi
uvicorn
streamlit
diffusers
torch
Pillow
requests
```

```
Writing requirements.txt
```

`requirements.txt` created.

✓ 3. `models.py`

Create a file named `models.py` in your project's root directory. This file will contain the logic for loading the `segmind/tiny-sd` model and generating images. Paste the following code into it:

```
import torch
from diffusers import AutoPipelineForText2Image
from PIL import Image

# Initialize a global variable to hold the pipeline
pipeline = None

def load_model():
    """Loads the segmind/tiny-sd pipeline."""
    global pipeline
    if pipeline is None:
        # Ensure we are using a device that can run the model efficiently
        # 'cuda' if GPU is available, otherwise 'cpu'
        device = "cuda" if torch.cuda.is_available() else "cpu"
        print(f"Loading model on device: {device}")
        pipeline = AutoPipelineForText2Image.from_pretrained("segmind/tiny-sd")
        pipeline.to(device)
    return pipeline

def generate_image(prompt: str, negative_prompt: str = None) -> Image.Image:
    """Generates an image from a text prompt using the loaded model.

    Args:
        prompt (str): The text prompt for image generation.
        negative_prompt (str, optional): The negative prompt to guide image generation.

    Returns:
        PIL.Image.Image: The generated image.
    """
    global pipeline
    if pipeline is None:
        load_model()

    print(f"Generating image for prompt: '{prompt}'")
    if negative_prompt:
        print(f"With negative prompt: '{negative_prompt}'")

    # Generate image. num_inference_steps can be adjusted for quality vs speed
    image_output = pipeline(prompt=prompt, negative_prompt=negative_prompt, num_inference_steps=50)
    return image_output

if __name__ == '__main__':
    # Example usage for testing the models.py file directly
    print("Testing models.py directly...")
    loaded_pipeline = load_model()
    test_image = generate_image("a cat playing guitar, high quality, realistic")
    test_image.save("test_image.png")
    print("Test image saved as test_image.png")
```

```

Flax classes are deprecated and will be removed in Diffusers v1.0.0. We r
Flax classes are deprecated and will be removed in Diffusers v1.0.0. We r
[transformers] `Siglip2ImageProcessorFast` is deprecated. The `Fast` suff
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_validators
warnings.warn(
Testing models.py directly...
Loading model on device: cuda
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:11
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your setting
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to acce
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please s
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthentic
model_index.json: 100% 584/584 [00:00<00:00, 46.5kB/s]

Download complete: 100% 1.06G/1.06G [00:13<00:00, 114MB/s]

Fetching 12 files: 100% 12/12 [00:13<00:00, 1.15s/it]

Loading pipeline components...: 100% 5/5 [00:00<00:00, 4.48it/s]
An error occurred while trying to fetch /root/.cache/huggingface/hub/mode
Defaulting to unsafe serialization. Pass `allow_pickle=False` to raise an
Loading weights: 100% 196/196 [00:00<00:00, 6744.03it/s]
An error occurred while trying to fetch /root/.cache/huggingface/hub/mode
Defaulting to unsafe serialization. Pass `allow_pickle=False` to raise an
Generating image for prompt: 'a cat playing guitar, high quality, realist
100% 25/25 [00:02<00:00, 13.02it/s]

Test image saved as test_image.png

```

```

%%writefile models.py
import torch
from diffusers import AutoPipelineForText2Image
from PIL import Image

# Initialize a global variable to hold the pipeline
pipeline = None

def load_model():
    """Loads the segmind/tiny-sd pipeline."""
    global pipeline
    if pipeline == None:
        # Ensure we are using a device that can run the model efficiently
        # 'cuda' if GPU is available, otherwise 'cpu'
        device = "cuda" if torch.cuda.is_available() else "cpu"
        print(f"Loading model on device: {device}")
        # For CPU, explicitly set torch_dtype to float32 for better compatibil
        # force_download=True is added to ensure all files are present, especia
        if device == "cpu":
            pipeline = AutoPipelineForText2Image.from_pretrained("segmind/tiny-
        else:
            pipeline = AutoPipelineForText2Image.from_pretrained("segmind/tiny-

```

```

        pipeline.to(device)
    return pipeline

def generate_image(prompt: str, negative_prompt: str = None) -> Image.Image:
    """Generates an image from a text prompt using the loaded model.

    Args:
        prompt (str): The text prompt for image generation.
        negative_prompt (str, optional): The negative prompt to guide image generation.

    Returns:
        PIL.Image.Image: The generated image.
    """
    global pipeline
    if pipeline == None:
        load_model()

    print(f"Generating image for prompt: '{prompt}'")
    if negative_prompt:
        print(f"With negative prompt: '{negative_prompt}'")

    # Generate image. num_inference_steps can be adjusted for quality vs speed
    image_output = pipeline(prompt=prompt, negative_prompt=negative_prompt, num_inference_steps=50)
    return image_output

if __name__ == '__main__':
    # Example usage for testing the models.py file directly
    print("Testing models.py directly...")
    loaded_pipeline = load_model()
    test_image = generate_image("a cat playing guitar, high quality, realistic")
    test_image.save("test_image.png")
    print("Test image saved as test_image.png")

```

Writing models.py

```

import importlib
import models

# Reload the models module to pick up the changes
importlib.reload(models)

print("Attempting to load model with torch.float32 on CPU (with force_download=True).")
try:
    # Reset the global pipeline to ensure a fresh load
    models.pipeline = None
    loaded_pipeline = models.load_model()
    if loaded_pipeline:
        print("Model loaded successfully on CPU with torch.float32.")
        print(f"Model device: {loaded_pipeline.device}")
    else:
        print("Model load failed (returned None).")
except Exception as e:
    print(f"An error occurred during model loading: {e}")

```

```

Attempting to load model with torch.float32 on CPU (with force_download=T
Loading model on device: cuda
model_index.json: 100% 584/584 [00:00<00:00, 59.8kB/s]
model_index.json: 100% 584/584 [00:00<00:00, 64.5kB/s]
Download complete: 0.00/0.00 [00:00<?, ?B/s]
Fetching 12 files: 100% 12/12 [00:00<00:00, 654.93it/s]
Loading pipeline components...: 100% 5/5 [00:00<00:00, 6.50it/s]
An error occurred while trying to fetch /root/.cache/huggingface/hub/mode
Defaulting to unsafe serialization. Pass `allow_pickle=False` to raise an
Loading weights: 100% 196/196 [00:00<00:00, 5577.76it/s]
An error occurred while trying to fetch /root/.cache/huggingface/hub/mode
Defaulting to unsafe serialization. Pass `allow_pickle=False` to raise an
Model loaded successfully on CPU with torch.float32.
Model device: cuda:0

```

`models.py` created.

✓ 4. `utils.py`

Create a file named `utils.py` in your project's root directory. This file will contain utility functions, specifically for converting a PIL Image into a byte stream without saving to disk. Paste the following code into it:

```

import io
from PIL import Image

def img_to_bytes(img: Image.Image, format: str = 'PNG') -> bytes:
    """Converts a PIL Image object to a byte array.

    Args:
        img (PIL.Image.Image): The input PIL Image.
        format (str): The image format for the byte stream (e.g., 'PNG', 'JPEG')

    Returns:
        bytes: The image as a byte array.
    """
    img_byte_arr = io.BytesIO()
    img.save(img_byte_arr, format=format)
    img_byte_arr.seek(0) # Rewind the buffer to the beginning
    return img_byte_arr.getvalue()

if __name__ == '__main__':
    # Example usage for testing utils.py directly
    print("Testing utils.py directly...")
    # Create a dummy image

```

```
dummy_image = Image.new('RGB', (100, 100), color = 'red')
image_bytes = img_to_bytes(dummy_image)
print(f"Generated {len(image_bytes)} bytes for the dummy image.")
# You could also try to open it again to verify
reloaded_image = Image.open(io.BytesIO(image_bytes))
print(f"Reloaded image size: {reloaded_image.size}")
```

Testing utils.py directly...
Generated 287 bytes for the dummy image.
Reloaded image size: (100, 100)

```
%%writefile utils.py
import io
from PIL import Image

def img_to_bytes(img: Image.Image, format: str = 'PNG') -> bytes:
    """Converts a PIL Image object to a byte array.

    Args:
        img (PIL.Image.Image): The input PIL Image.
        format (str): The image format for the byte stream (e.g., 'PNG', 'JPEG')

    Returns:
        bytes: The image as a byte array.
    """
    img_byte_arr = io.BytesIO()
    img.save(img_byte_arr, format=format)
    img_byte_arr.seek(0) # Rewind the buffer to the beginning
    return img_byte_arr.getvalue()

if __name__ == '__main__':
    # Example usage for testing utils.py directly
    print("Testing utils.py directly...")
    # Create a dummy image
    dummy_image = Image.new('RGB', (100, 100), color = 'red')
    image_bytes = img_to_bytes(dummy_image)
    print(f"Generated {len(image_bytes)} bytes for the dummy image.")
    # You could also try to open it again to verify
    reloaded_image = Image.open(io.BytesIO(image_bytes))
    print(f"Reloaded image size: {reloaded_image.size}")
```

Writing utils.py

utils.py created.

✓ 5. main.py (FastAPI Backend)

Create a file named `main.py` in your project's root directory. This file will set up the FastAPI server with an endpoint to generate images. Paste the following code into it.

To run this FastAPI app locally, navigate to your project directory in a terminal and execute: `uvicorn main:app --reload`

```
from fastapi import FastAPI, Response, Query
from models import generate_image, load_model # Assuming models.py is in the same directory
from utils import img_to_bytes # Assuming utils.py is in the same directory
from typing import Optional
import uvicorn
import time

app = FastAPI(
    title="Text-to-Image Generation Service",
    description="A FastAPI service for generating images using segmind/tiny-sd",
    version="1.0.0",
)

# Load the model when the FastAPI application starts up
@app.on_event("startup")
async def startup_event():
    print("Loading Stable Diffusion model during startup...")
    load_model()
    print("Model loaded successfully.")

@app.get("/generate/image", response_class=Response, tags=["Image Generation"])
async def generate_image_endpoint(
    prompt: str = Query(..., description="The text prompt for image generation"),
    negative_prompt: Optional[str] = Query(None, description="Optional negative prompt"),
):
    """Generates an image from the provided text prompt.

    Args:
        prompt (str): The text prompt for image generation.
        negative_prompt (str, optional): An optional negative prompt.

    Returns:
        Response: An image file in PNG format.
    """
    start_time = time.time()
    try:
        pil_image = generate_image(prompt, negative_prompt)
        image_bytes = img_to_bytes(pil_image, format='PNG')
        end_time = time.time()
        print(f"Image generation and conversion took {end_time - start_time:.2f} seconds")
        return Response(content=image_bytes, media_type="image/png")
    except Exception as e:
        print(f"Error during image generation: {e}")
        return Response(content=f"Error generating image: {e}", status_code=500)

if __name__ == '__main__':
    # This block allows you to run the app directly using `python main.py` for
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

```
/tmp/ipykernel_1685/2901815554.py:15: DeprecationWarning:
    on_event is deprecated, use lifespan event handlers instead.
```

Read more about it in the
[FastAPI docs for Lifespan Events](<https://fastapi.tiangolo.com/a>

```
@app.on_event("startup")
```

```
-----
---
RuntimeError                                Traceback (most recent call
last)
/tmp/ipykernel_1685/2901815554.py in <cell line: 0>()
    46 if __name__ == '__main__':
    47     # This block allows you to run the app directly using
`python main.py` for development
---> 48     uvicorn.run(app, host="0.0.0.0", port=8000)

-----  2 frames -----
/usr/lib/python3.12/asyncio/runners.py in run(main, debug, loop_factory)
    189     if events._get_running_loop() is not None:
    190         # fail fast with short traceback
--> 191         raise RuntimeError(
    192             "asyncio.run() cannot be called from a running event
loop")
    193
```

RuntimeError: asyncio.run() cannot be called from a running event loop

Next steps: [Explain error](#)

```
%%writefile main.py
from fastapi import FastAPI, Response, Query
from models import generate_image, load_model # Assuming models.py is in the s
from utils import img_to_bytes # Assuming utils.py is in the same directory
from typing import Optional
import uvicorn
import time

app = FastAPI(
    title="Text-to-Image Generation Service",
    description="A FastAPI service for generating images using segmind/tiny-sd
    version="1.0.0",
)

# Load the model when the FastAPI application starts up
@app.on_event("startup")
async def startup_event():
    print("Loading Stable Diffusion model during startup...")
    load_model()
    print("Model loaded successfully.")
```



```

@app.get("/generate/image", response_class=Response, tags=["Image Generation"])
async def generate_image_endpoint(
    prompt: str = Query(..., description="The text prompt for image generation"),
    negative_prompt: Optional[str] = Query(None, description="Optional negative prompt"):
    """Generates an image from the provided text prompt.

    Args:
        prompt (str): The text prompt for image generation.
        negative_prompt (str, optional): An optional negative prompt.

    Returns:
        Response: An image file in PNG format.
    """
    start_time = time.time()
    try:
        pil_image = generate_image(prompt, negative_prompt)
        image_bytes = img_to_bytes(pil_image, format='PNG')
        end_time = time.time()
        print(f"Image generation and conversion took {end_time - start_time:.2f} seconds")
        return Response(content=image_bytes, media_type="image/png")
    except Exception as e:
        print(f"Error during image generation: {e}")
        return Response(content=f"Error generating image: {e}", status_code=500)

if __name__ == '__main__':
    # This block allows you to run the app directly using `python main.py` for
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

`main.py` created.

✓ 6. `client.py` (Streamlit Frontend)

Create a file named `client.py` in your project's root directory. This file will be your Streamlit application, providing an interactive UI to interact with your FastAPI backend. Paste the following code into it.

To run this Streamlit app locally, open a new terminal in your project directory (while `main.py` is still running) and execute: `streamlit run client.py`

Ensure your FastAPI server (`main.py`) is running on `http://localhost:8000` before starting the Streamlit client.

```

import streamlit as st
import requests
import io

# FastAPI server URL
FASTAPI_URL = "http://localhost:8000"

st.set_page_config(page_title="DTA Creative AI Service", layout="wide")

```

```

st.title("🌟 Discovery-to-Action Creative AI Service")
st.markdown("Generate images using `segmind/tiny-sd` via a FastAPI backend.")

# Initialize chat history
if "messages" not in st.session_state:
    st.session_state.messages = []

# Display chat messages from history on app rerun
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        if message["type"] == "text":
            st.markdown(message["content"])
        elif message["type"] == "image":
            st.image(message["content"], caption=message["caption"], use_column_width=True)

# React to user input
if prompt := st.chat_input("Enter your image prompt here..."):
    # Display user message in chat message container
    with st.chat_message("user"):
        st.markdown(prompt)
    # Add user message to chat history
    st.session_state.messages.append({"role": "user", "type": "text", "content": prompt})

    # Get optional negative prompt from a sidebar input (or a collapsed expander)
    # For this example, we'll keep it simple and just use the prompt.
    # In a real app, you might have an expander for advanced options.
    negative_prompt_input = ""
    with st.expander("Advanced Options (Negative Prompt)"):
        negative_prompt_input = st.text_input("Negative Prompt (e.g., blurry, low quality)")

    # Send request to FastAPI backend
    with st.spinner("Generating image..."):
        params = {"prompt": prompt}
        if negative_prompt_input:
            params["negative_prompt"] = negative_prompt_input

    try:
        response = requests.get(f"{FASTAPI_URL}/generate/image", params=params)

        if response.status_code == 200:
            image_bytes = response.content
            # Display assistant response with image
            with st.chat_message("assistant"):
                st.image(image_bytes, caption=f"Generated for: '{prompt}'")
            # Add assistant response to chat history
            st.session_state.messages.append({"role": "assistant", "type": "image", "content": image_bytes, "caption": f"Generated for: '{prompt}'"})
        else:
            error_message = response.text or f"Error: {response.status_code}"
            with st.chat_message("assistant"):
                st.error(f"Failed to generate image: {error_message}")
            st.session_state.messages.append({"role": "assistant", "type": "text", "content": error_message})
    except requests.exceptions.ConnectionError:
        with st.chat_message("assistant"):
            st.error(f"Could not connect to FastAPI server at {FASTAPI_URL}")
        st.session_state.messages.append({"role": "assistant", "type": "text", "content": f"Could not connect to FastAPI server at {FASTAPI_URL}"})
    except Exception as e:
        with st.chat_message("assistant"):
            st.error(f"Unexpected error: {e}")
        st.session_state.messages.append({"role": "assistant", "type": "text", "content": f"Unexpected error: {e}"})

```

```
st.error(f"An unexpected error occurred: {e}")
st.session_state.messages.append({"role": "assistant", "type": "text", "content": "An unexpected error occurred: " + str(e)})
```

```
%%writefile client.py
import streamlit as st
import requests
import io

# FastAPI server URL
FASTAPI_URL = "http://localhost:8000"

st.set_page_config(page_title="DTA Creative AI Service", layout="wide")

st.title("🌟 Discovery-to-Action Creative AI Service")
st.markdown("Generate images using `segmind/tiny-sd` via a FastAPI backend.")

# Initialize chat history
if "messages" not in st.session_state:
    st.session_state.messages = []

# Display chat messages from history on app rerun
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        if message["type"] == "text":
            st.markdown(message["content"])
        elif message["type"] == "image":
            st.image(message["content"], caption=message["caption"], use_column_width=True)

# React to user input
if prompt := st.chat_input("Enter your image prompt here..."):
    # Display user message in chat message container
    with st.chat_message("user"):
        st.markdown(prompt)
    # Add user message to chat history
    st.session_state.messages.append({"role": "user", "type": "text", "content": prompt})

    # Get optional negative prompt from a sidebar input (or a collapsed expander)
    # For this example, we'll keep it simple and just use the prompt.
    # In a real app, you might have an expander for advanced options.
    negative_prompt_input = ""
    with st.expander("Advanced Options (Negative Prompt)"):
        negative_prompt_input = st.text_input("Negative Prompt (e.g., blurry, low quality)")

    # Send request to FastAPI backend
    with st.spinner("Generating image..."):
        params = {"prompt": prompt}
        if negative_prompt_input:
            params["negative_prompt"] = negative_prompt_input

    try:
        response = requests.get(f"{FASTAPI_URL}/generate/image", params=params)

        if response.status_code == 200:
            image_bytes = response.content
            # Display assistant response with image
            with st.chat_message("assistant"):
                st.image(image_bytes, caption=f"Generated for: '{prompt}'")
            # Add assistant response to chat history
            st.session_state.messages.append({"role": "assistant", "type": "image", "content": image_bytes, "caption": f"Generated for: '{prompt}'"})
```

```

else:
    error_message = response.text or f"Error: {response.status_code}"
    with st.chat_message("assistant"):
        st.error(f"Failed to generate image: {error_message}")
        st.session_state.messages.append({"role": "assistant", "type": "text"})
except requests.exceptions.ConnectionError:
    with st.chat_message("assistant"):
        st.error(f"Could not connect to FastAPI server at {FASTAPI_URL}")
        st.session_state.messages.append({"role": "assistant", "type": "text"})
except Exception as e:
    with st.chat_message("assistant"):
        st.error(f"An unexpected error occurred: {e}")
        st.session_state.messages.append({"role": "assistant", "type": "text"})

```

✓ `client.py` created.

All necessary files have been created in your Colab environment. Now, let's run the FastAPI backend and Streamlit frontend.

```

# Start the FastAPI server in the background
print("Starting FastAPI backend...")
get_ipython().system_raw('python -m uvicorn main:app --host 0.0.0.0 --port 8000')
print("FastAPI backend started on port 8000. It is running in the background and ready to receive requests.")

```

```

# Install localtunnel to expose the Streamlit app to a public URL
!npm install -g localtunnel

# Start the Streamlit app in the background and expose it using localtunnel
print("Starting Streamlit frontend and creating a public URL...")
# Run Streamlit in the background, redirecting its output.
# Then, use localtunnel to get a public URL for Streamlit (port 8501).
get_ipython().system_raw('streamlit run client.py --server.port 8501 --server.headless true')

# Give some time for localtunnel to create the URL
import time
time.sleep(10)

# Read the public URL from the file
with open('/content/lt.url') as f:
    public_url = f.read().strip()

print(f"\nStreamlit app is running! Access it via this public URL: {public_url}")
print("You may need to click 'Click to Continue' on the localtunnel page if it shows up.")

```

You should now see a public URL above. Click on that link to open the Streamlit application in a new tab. The Streamlit app will connect to the FastAPI backend running in this Colab instance to generate images.

Next Steps & Instructions:

1. **Save the files:** Copy the content from each of the above cells into their respective `.py` files (`requirements.txt`, `models.py`, `utils.py`, `main.py`, `client.py`) in a new local project directory.

2. **Create a Virtual Environment (Recommended):**

```
python -m venv venv  
source venv/bin/activate # On Windows: .\venv\Scripts\activate
```

3. **Install dependencies locally:**

```
pip install -r requirements.txt
```

4. **Run the FastAPI Backend:** Open your first terminal, navigate to your project directory, and run:

```
uvicorn main:app --reload
```

(The `--reload` flag is useful for development as it restarts the server on code changes).

5. **Run the Streamlit Frontend:** Open a **second terminal**, navigate to your project directory, and run:

```
streamlit run client.py
```

6. **Access the Streamlit App:** Streamlit will provide you with a local URL (usually `http://localhost:8501`) to access the application in your web browser.

This setup directly addresses all the points from the project brief and the feedback you received. Remember to also create a `README.md` file as per the project submission requirements, documenting your environment, architecture, latency findings, negative prompt explanation, and marketing workflow proposal.

Proposed GitHub Repository Name and Description

Repository Name: `Creative-AI-Service-Text-to-Image`

Description: A full-stack text-to-image AI service using FastAPI for the backend and Streamlit for the interactive frontend, powered by Hugging Face Diffusers.

✓ Creative AI Service: Text-to-Image Generator

This project demonstrates a full-stack application for generating images from text prompts using a lightweight Stable Diffusion model. It leverages FastAPI for the backend API, Streamlit for an interactive web-based frontend, and Hugging Face's `diffusers` library with the `segmind/tiny-sd` model for image generation.

Table of Contents

1. [Project Overview](#)
2. [Features](#)
3. [Architecture](#)
4. [Local Setup and Installation](#)
5. [Running the Application](#)
6. [Project Structure](#)
7. [Model Details & Latency](#)
8. [Understanding Negative Prompts](#)
9. [Marketing Workflow Proposal](#)

1. Project Overview

This application provides a user-friendly interface to generate images based on textual descriptions (prompts). It's designed for rapid prototyping and demonstration of text-to-image AI capabilities, specifically tailored for scenarios requiring a FastAPI backend and a Streamlit frontend.

2. Features

- **Text-to-Image Generation:** Generate unique images from text prompts.
- **Negative Prompt Support:** Guide image generation away from undesired features.

- **FastAPI Backend:** Robust and scalable API for serving the image generation model.
- **Streamlit Frontend:** Interactive and intuitive user interface.
- **Containerization Ready:** Designed with separate files for easy containerization.
- **Colab Compatibility:** Demonstrates deployment and execution within a Google Colab environment.

3. Architecture

The application follows a client-server architecture:

- **Frontend (Streamlit `client.py`):** Provides the web interface where users input prompts and view generated images. It communicates with the FastAPI backend to request image generation.
- **Backend (FastAPI `main.py`):** Exposes a `/generate/image` API endpoint. It handles requests from the Streamlit client, loads the `segmind/tiny-sd` model (via `models.py`), generates the image, and returns it as a PNG byte stream.
- **Model Logic (`models.py`):** Encapsulates the logic for loading the `segmind/tiny-sd` model and performing image generation using the `diffusers` library.
- **Utility Functions (`utils.py`):** Contains helper functions, specifically for converting PIL Image objects to byte streams for API responses.

```
graph TD
    User --> Streamlit[Streamlit Frontend (client.py)]
    Streamlit -- HTTP GET /generate/image --> FastAPI[FastAPI Backend (main.py)]
    FastAPI --> Models[models.py (Image Generation)]
    Models -- Uses --> Diffusers[Hugging Face Diffusers]
    Models -- Loads --> TinySD[segmind/tiny-sd Model]
    Models -- Returns PIL Image --> FastAPI
    FastAPI -- Converts PIL Image to Bytes --> Utils[utils.py]
    Utils -- Returns Image Bytes --> FastAPI
    FastAPI -- Returns Image Bytes --> Streamlit
    Streamlit -- Displays Image --> User
```

4. Local Setup and Installation

To run this project locally, follow these steps:

1. **Clone the Repository:**

```
git clone <repository-url>
cd Creative-AI-Service-Text-to-Image
```

2. Create a Virtual Environment (Recommended):

```
python -m venv venv
# On macOS/Linux:
source venv/bin/activate
# On Windows:
.\venv\Scripts\activate
```

3. Install Dependencies:

```
pip install -r requirements.txt
```

The `requirements.txt` file contains:

```
fastapi
uvicorn
streamlit
diffusers
torch
Pillow
requests
```

5. Running the Application

Ensure your virtual environment is activated before running the following commands.

1. **Start the FastAPI Backend:** Open your first terminal, navigate to your project directory, and run:

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

(The `--reload` flag is useful for development as it automatically restarts the server on code changes).

2. **Start the Streamlit Frontend:** Open a **second terminal**, navigate to your project directory, and run:

```
streamlit run client.py
```


3. **Access the Streamlit App:** Streamlit will provide you with a local URL (usually `http://localhost:8501`) to access the application in your web browser.

Make sure the FastAPI backend is running before starting the Streamlit client.

6. Project Structure

```
. # Project Root
├── main.py          # FastAPI backend application
├── models.py        # Contains AI model loading and inference logic
├── utils.py         # Utility functions (e.g., image to bytes conversion)
├── client.py        # Streamlit frontend application
└── requirements.txt # List of Python dependencies
```

7. Model Details & Latency

- **Model Used:** `segmind/tiny-sd` is a distilled version of Stable Diffusion, offering faster inference with a smaller footprint. It's ideal for applications where speed and resource efficiency are critical.
- **Inference Steps:** The `generate_image` function uses `num_inference_steps=25` to balance image quality and generation speed. This can be adjusted based on desired output quality and performance requirements.

Latency Findings (Conceptual - requires actual measurement): Upon running the application locally or in a Colab environment, observe the time taken for image generation. The `main.py` backend logs the `Image generation and conversion took X.XX seconds.` message. For `segmind/tiny-sd` with 25 inference steps on typical GPU hardware (e.g., Colab T4 GPU), generation times are expected to be in the range of 1-5 seconds, significantly faster than larger Stable Diffusion models.

8. Understanding Negative Prompts

A **negative prompt** is an additional text input that guides the image generation process *away* from certain features or styles. While a positive prompt tells the model what you *want* to see, a negative prompt tells it what you *don't* want to see.

Example:

- **Positive Prompt:** "A beautiful landscape, vibrant colors, clear sky."
- **Negative Prompt:** "Blurry, low quality, grayscale, deformed, bad anatomy, text, watermark."

Using negative prompts can significantly improve the quality and relevance of generated images by preventing common artifacts or undesired elements.

9. Marketing Workflow Proposal

This Creative AI Service can be integrated into various marketing workflows to generate unique visual content efficiently.

1. Content Generation for Social Media/Blogs:

- **Idea:** Quickly generate featured images for blog posts, social media updates, or campaign visuals.
- **Workflow:** Content creator inputs a description of the desired image (e.g., "futuristic city skyline at sunset, cyberpunk aesthetic"). The AI generates several options, which can then be refined with negative prompts or used as inspiration for further iteration. This drastically reduces reliance on stock photos or manual graphic design for initial concepts.

2. Rapid Prototyping for Ad Campaigns:

- **Idea:** Visualize different ad concepts without engaging designers for every initial idea.
- **Workflow:** Marketing team brainstorms various ad headlines and associated visual concepts. Prompts are entered into the tool to generate visual mockups (e.g., "a happy family enjoying a new gadget, bright lighting, product in focus"). These visuals can be used for internal review, A/B testing ideas, or even for preliminary client presentations.

3. Personalized Marketing Visuals:

- **Idea:** Create unique images for targeted email campaigns or personalized landing pages.
- **Workflow:** Based on customer segmentation or preference data, generate bespoke images (e.g., "a serene beach scene with a person meditating" for a wellness product, or "a bustling urban coffee shop" for a city-centric service). This helps in making marketing messages more engaging and relevant to individual users.

4. Mood Board Creation and Concept Exploration:

- **Idea:** Quickly create visual mood boards for new product launches, brand refreshes, or event themes.
- **Workflow:** Input broad themes or keywords to generate a collection of images that convey a certain aesthetic or feeling. This aids in early-stage ideation and ensures

everyone on the team is aligned on the visual direction.

Benefits:

- **Speed:** Generate visuals in seconds, not hours or days.
- **Cost-Effective:** Reduce expenses associated with stock photography licenses or external design services for initial concepts.
- **Creativity:** Explore a wider range of visual ideas and push creative boundaries.
- **Iterative Design:** Easily refine concepts through prompt adjustments and negative prompts.

```
print("Killing existing FastAPI (uvicorn) and Streamlit processes...")
# Kill any running uvicorn processes
!kill $(ps aux | grep 'uvicorn main:app' | grep -v 'grep' | awk '{print $2}')
# Kill any running streamlit processes
!kill $(ps aux | grep 'streamlit run client.py' | grep -v 'grep' | awk '{print $2}')
# Kill any running localtunnel processes
!kill $(ps aux | grep 'npx localtunnel' | grep -v 'grep' | awk '{print $2}')
print("Existing processes terminated (if any).")
```

```
# Start the FastAPI server in the background
print("Starting FastAPI backend...")
get_ipython().system_raw('python -m uvicorn main:app --host 0.0.0.0 --port 8000')
print("FastAPI backend started on port 8000. It is running in the background and ready to accept requests.")
```

```
import time
import os
import re

# Install localtunnel to expose the Streamlit app to a public URL (if not already installed)
print("Installing localtunnel...")
!npm install -g localtunnel

# Clear previous log file if it exists, to ensure we get a fresh output
log_file_path = '/content/lt_raw.log'
if os.path.exists(log_file_path):
    os.remove(log_file_path)
# Also clear the old lt.url file if it exists from previous attempts
if os.path.exists('/content/lt.url'):
    os.remove('/content/lt.url')

print("Starting Streamlit frontend in the background...")
get_ipython().system_raw('streamlit run client.py --server.port 8501 --server.headless true')

print("Starting localtunnel in the background and capturing its output...")
# Run localtunnel and redirect all its output (stdout and stderr) to a log file
get_ipython().system_raw(f'npx localtunnel --port 8501 > {log_file_path} 2>&1 &')

# Give some time for localtunnel to establish the connection and write output
initial_wait_time = 30
print(f"Waiting {initial_wait_time} seconds for localtunnel to start and write output.")
time.sleep(initial_wait_time)
```

```

public_url = ''
if os.path.exists(log_file_path):
    with open(log_file_path, 'r') as f:
        log_content = f.read()
        print(f"Captured localtunnel log content:\n---\n{log_content}\n---")

    # Regex to find the URL in the localtunnel output
    # The URL typically looks like "your url is: https://<subdomain>.localtunnel.me"
    # Using \S+ to match one or more non-whitespace characters for the URL
    match = re.search(r'your url is: (https://\S+)', log_content)
    if match:
        public_url = match.group(1).strip()
    else:
        print("No URL found in log content.")
else:
    print(f"Localtunnel log file '{log_file_path}' did not exist after initial")

# Fallback if the URL was not found initially
if not public_url:
    print("URL not found in initial log. Retrying after more sleep to check for")
    retry_wait_time = 30
    print(f"Waiting an additional {retry_wait_time} seconds...")
    time.sleep(retry_wait_time)
    if os.path.exists(log_file_path):
        with open(log_file_path, 'r') as f:
            log_content = f.read()
            print(f"Captured localtunnel log content after retry:\n---\n{log_content}\n---")
            match = re.search(r'your url is: (https://\S+)', log_content)
            if match:
                public_url = match.group(1).strip()
            else:
                print("No URL found in log content after retry.")
    else:
        print(f"Localtunnel log file '{log_file_path}' did not exist after retry")

if public_url:
    print(f"\nStreamlit app is running! Access it via this NEW public URL: {public_url}")
    print("You may need to click 'Click to Continue' on the localtunnel page if it's not visible")
else:
    print(f"\nFailed to get a public URL from localtunnel after multiple attempts")

```

Restarting services to integrate updated model logic and ensure fresh connections.

```

print("Killing existing FastAPI (uvicorn) and Streamlit processes...")
# Kill any running uvicorn processes
!kill $(ps aux | grep 'uvicorn main:app' | grep -v 'grep' | awk '{print $2}')
# Kill any running streamlit processes
!kill $(ps aux | grep 'streamlit run client.py' | grep -v 'grep' | awk '{print $2}')
# Kill any running localtunnel processes
!kill $(ps aux | grep 'npx localtunnel' | grep -v 'grep' | awk '{print $2}')
print("Existing processes terminated (if any).")

```

```
# Start the FastAPI server in the background
print("Starting FastAPI backend with updated model logic...")
get_ipython().system_raw('python -m uvicorn main:app --host 0.0.0.0 --port 8000')
print("FastAPI backend started on port 8000. It is running in the background and ready to receive requests.")
```

```
import time
import os
import re

# Ensure localtunnel is installed (needed if previous process was killed or if not installed)
print("Installing localtunnel...")
!npm install -g localtunnel

# Clear previous log file to ensure we get a fresh output
log_file_path = '/content/lt_raw.log'
if os.path.exists(log_file_path):
    os.remove(log_file_path)

print("Starting Streamlit frontend in the background...")
get_ipython().system_raw('streamlit run client.py --server.port 8501 --server.headless True')

print("Starting localtunnel in the background and capturing its output...")
# Run localtunnel and redirect all its output (stdout and stderr) to a log file
get_ipython().system_raw(f'npx localtunnel --port 8501 > {log_file_path} 2>&1 &')

# Give some time for localtunnel to establish the connection and write output
initial_wait_time = 30
print(f"Waiting {initial_wait_time} seconds for localtunnel to start and write output.")
time.sleep(initial_wait_time)

public_url = ''
if os.path.exists(log_file_path):
    with open(log_file_path, 'r') as f:
        log_content = f.read()
    print(f"Captured localtunnel log content:\n---\n{log_content}\n---")

    # Regex to find the URL in the localtunnel output
    match = re.search(r'your url is: (https://\S+)', log_content)
    if match:
        public_url = match.group(1).strip()
    else:
        print("No URL found in log content.")
else:
    print(f"Localtunnel log file '{log_file_path}' did not exist after initial wait.")

# Fallback if the URL was not found initially
if not public_url:
    print("URL not found in initial log. Retrying after more sleep to check for output.")
    retry_wait_time = 30
    print(f"Waiting an additional {retry_wait_time} seconds...")
    time.sleep(retry_wait_time)
    if os.path.exists(log_file_path):
        with open(log_file_path, 'r') as f:
            log_content = f.read()
        print(f"Captured localtunnel log content after retry:\n---\n{log_content}\n---")
        match = re.search(r'your url is: (https://\S+)', log_content)
        if match:
            public_url = match.group(1).strip()
    else:
        print("Localtunnel log file still does not exist after retry.")
```